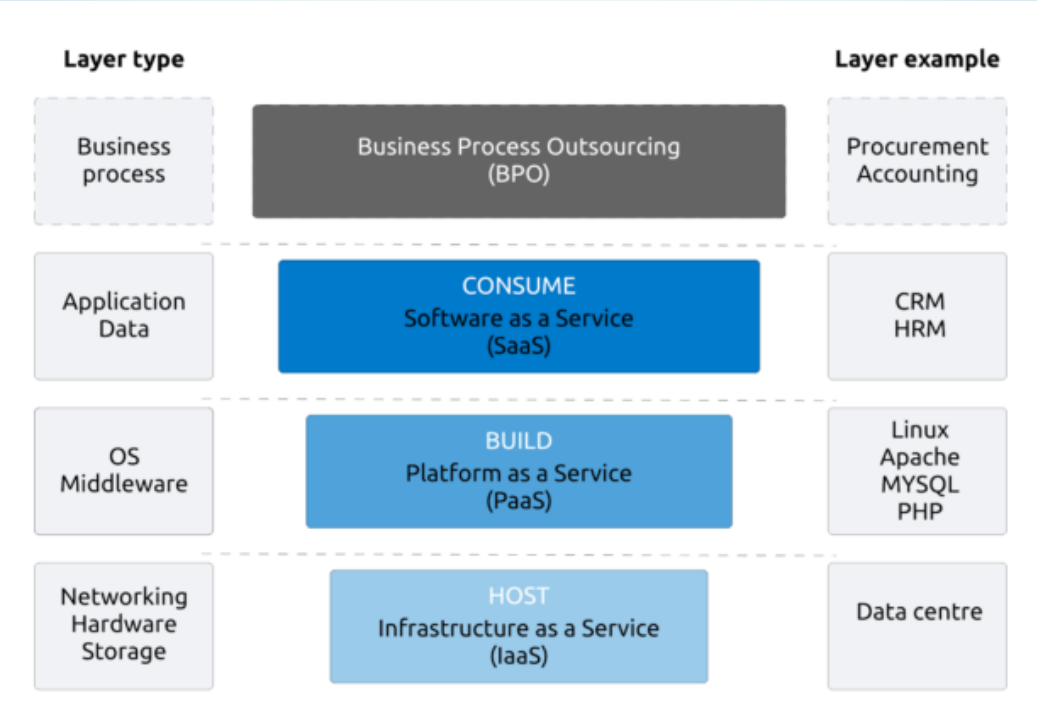
Module 3
Cloud Architecture

Content

- Cloud Architecture- Layers and Models :Architecture for Cloud Application
- Cloud Application requirements
- SOA for Cloud applications
- Parallelization within Cloud Applications
- Multi-ties Application Architecture
- Study of Cloud Tools and Platforms

Cloud Architecture- Layers and Models



Cloud Architecture- Layers and Models

- **Infrastructure as a Service (IaaS)**
  - The basic layer of cloud is the infrastructure –IaaS (Infrastructure as a service). This layer is basically **hardware and network**.
  - What distinguishes this from a regular server or hosting company are mainly two things: **scalability and virtualisation**.
  - IaaS service providers scale this layer in such a manner that the additional cost of adding more storage or bandwidth is minimal.
  - Owing to virtualisation, these providers are able to use up to 90% of their computing resources in contrast to traditional hosting services where servers may lay idle at times.
  - One of the first services where this was seen was in Gmail, the Google email service where each user gets around 8 GB of free storage. In order to do this, Google built gigantic data centres consisting of (probably) millions of servers.

Cloud Architecture- Layers and Models

Key Features of IaaS

- Highly scalable resources
- Enterprise-grade infrastructure
- Cost depends on consumption
- Multitenant architecture, i.e. a single piece of hardware serves many users
- The client gets complete control over the infrastructure

IaaS Advantages

- The most flexible and dynamic model
- Cost-effective due to pay-as-you-go pricing
- Easy to use due to the automated deployment of hardware
- Management tasks are virtualized, so employees have more free time for other tasks

IaaS Disadvantages

- Data security issues due to multitenant architecture
- Vendor outages make customers unable to access their data for a while
- The need for team training to learn how to manage new infrastructure

Cloud Architecture- Layers and Models

- **Platform as a Service (PaaS)**
  - The second layer of the cloud is the platform – the PaaS (Platform as a service). The platform layer provides resources to actually **build applications**.
  - In combination with IaaS, PaaS provides the ability to develop, test, run, and host applications.
  - This layer of the cloud caters to the requirements of software developers as it is the place where new applications are developed.
  - You can use PaaS services to build and test your applications on the cloud before deploying them.
  - In fact, it is designed in such a manner that it supports the entire lifecycle of an application, right from building, testing and deployment to maintenance and updating. Like IaaS, PaaS includes infrastructure, but also includes development tools, database management systems and much more.
  - An example of a well-known PaaS is Microsoft Azure. This platform provides developers with swift access to a full development and deployment environment and even let you host the application you are building.

Cloud Architecture- Layers and Models

Key Features of PaaS

- Allows for developing, testing and hosting apps in the same environment
- Resources can be scaled up and down depending on business needs
- Multiple users can access the same app in development
- The user doesn't have complete control over the infrastructure
- Web services and databases are integrated
- Remote teams can collaborate easily

PaaS Advantages

- PaaS-built software is highly scalable, available and multi-tenant, as it is cloud-based
- The development process is quickened and simplified
- Reduced expenses for creating, testing and launching apps
- Automated company policy
- Reduced amount of coding required
- Allows for easy migrating to the hybrid cloud

PaaS Disadvantages

- Data security issues
- Compatibility of existing infrastructure (not every element can be cloud-enabled)
- Dependency on vendor's speed, reliability and support

Cloud Architecture- Layers and Models

- **Software as a Service (SaaS)**
  - The third cloud layer is the actual Software – the SaaS (Software as a service). SaaS has been used for many years, but in a Cloud setting, it is the layer in which the user consumes the offering from the service provider.
  - Here, organizations rent the usage of a SaaS application, and users connect to it by means of the internet.
  - The application, therefore, needs to be web-based server so that it can be accessed from anywhere.
  - In this case, the service provider offers both the software and the hardware.
  - The SaaS layer must be web-based and hence accessible from everywhere and preferably on any device. The key is to understand that it makes no sense to ask whether a service is cloud or SaaS, as SaaS is a layer in the cloud stack.

Cloud Architecture- Layers and Models

Key Features

- The subscription model of utilizing
- No need to download, install or upgrade software
- Resources can be scaled depending on requirements
- Apps are accessible from any connected device
- The provider is responsible for everything

SaaS Advantages

- No hardware costs
- No initial setup costs
- Automated upgrades
- Cross-device compatibility
- Accessible from any location
- Pay-as-you-go model
- Scalability
- Easy customization

SaaS Disadvantages

- Loss of control
- Limited range of solutions
- Connectivity is a must

Cloud Architecture- Layers and Models

Service Provider

- A cloud service provider is a third-party company offering a cloud-based platform, infrastructure, application or storage services. Much like a homeowner would pay for a utility such as electricity or gas, companies typically have to pay only for the amount of cloud services they use, as business demands require.
- Besides the pay-per-use model, cloud service providers also give companies a wide range of benefits. Businesses can take advantage of scalability and flexibility by not being limited to physical constraints of on-premises servers, the reliability of multiple data centers with multiple redundancies, customisation by configuring servers to your preferences and responsive load balancing which can easily respond to changing demands.
- Though businesses should also evaluate security considerations of storing information in the cloud to ensure industry-recommended access and compliance management configurations and practices are enacted and met.

Cloud Application Requirements

Step 1: Requirement Analysis

- Identify the business problem and objectives.
- Determine user requirements and expected services.
- Analyze workload characteristics and usage patterns.
- Example: An online learning platform needs video streaming, user authentication, and assessment modules.

Step 2: Scalability Requirement

- The application should support increasing or decreasing workloads.
- Resources must be allocated dynamically based on demand.
- Example: During online examinations, the system should automatically add servers to handle **more users.**

Step 3: Availability Requirement

- The application should remain accessible with minimal downtime.
- Redundant resources and failover mechanisms should be implemented.
- Example: E-commerce websites must remain available during festive sales.

Step 4: Reliability Requirement

- The system should perform consistently without failures.
- Data integrity and fault tolerance mechanisms should be incorporated.
- Example: Banking applications must ensure every transaction is completed accurately.

Cloud Application Requirements

Step 5: Performance Requirement

- Applications should provide fast response times and efficient processing.
- Load balancing and resource optimization should be used.
- Example: Search results on a shopping website should appear within seconds.

Step 6: Security Requirement

- Protect data, applications, and infrastructure from unauthorized access.
- Implement authentication, authorization, encryption, and auditing.
- Example: Cloud-based healthcare systems must secure patient records.

Step 7: Data Management Requirement

- Efficient storage, retrieval, backup, and recovery of data.
- Support for distributed databases and big data processing.
- Example: Social media platforms store and manage billions of user posts and images.

Step 8: Interoperability Requirement

- The application should work across different cloud platforms and services.
- Standard APIs and protocols should be used.
- Example: An application deployed on AWS should be able to interact with services hosted on Azure.

Cloud Application Requirements

Step 9: Elasticity Requirement

- Resources should be automatically provisioned and released based on demand.
- Pay only for resources actually used.
- Example: Video streaming services allocate additional resources during live events.

Step 10: Cost Efficiency Requirement

- Optimize resource utilization to reduce operational costs.Select appropriate cloud service models (IaaS, PaaS, SaaS).
- Example: Startups often use SaaS solutions to avoid infrastructure costs.

Step 11: Compliance and Governance Requirement

- Ensure adherence to legal, regulatory, and organizational policies.
- Maintain audit trails and data privacy standards.
- Example: Financial institutions must comply with RBI and data protection regulations.

Step 12: Monitoring and Maintenance Requirement

- Continuously monitor application performance and resource usage.
- Implement logging, alerting, and automated maintenance.
- Example: Cloud monitoring tools detect server failures and notify administrators immediately.

CLOUD APPLICATION REQUIREMENTS

Step-by-Step Guide

1 REQUIREMENT ANALYSIS

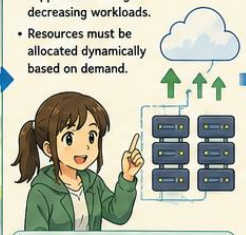
- Identify the business problem and objectives.
- Determine user requirements and expected services.
- Analyze workload characteristics and usage patterns.



Example: An online learning platform needs video streaming, user authentication, and assessment modules.

2 SCALABILITY REQUIREMENT

- The application should support increasing or decreasing workloads.
- Resources must be allocated dynamically based on demand.



Example: During online examinations, the system should automatically add servers to handle more users.

3 AVAILABILITY REQUIREMENT

- The application should remain accessible with minimal downtime.
- Redundant resources and failover mechanisms should be implemented.



Example: E-commerce websites must remain available during festive sales.

4 RELIABILITY REQUIREMENT

- The system should perform consistently without failures.
- Data integrity and fault tolerance mechanisms should be incorporated.



Example: Banking applications must ensure every transaction is completed accurately.

5 PERFORMANCE REQUIREMENT

- Applications should provide fast response times and efficient processing.
- Load balancing and resource optimization should be used.



Example: Search results on a shopping website should appear within seconds.

6 SECURITY REQUIREMENT

- Protect data, applications, and infrastructure from unauthorized access.
- Implement authentication, authorization, encryption, and auditing.



Example: Cloud-based healthcare systems must secure patient records.

7 DATA MANAGEMENT REQUIREMENT

- Efficient storage, retrieval, backup, and recovery of data.
- Support for distributed databases and big data processing.



Example: Social media platforms store and manage billions of user posts and images.

8 INTEROPERABILITY REQUIREMENT

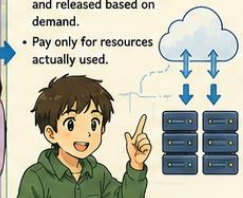
- The application should work across different cloud platforms and services.
- Standard APIs and protocols should be used.



Example: An application deployed on AWS should be able to interact with services hosted on Azure.

9 ELASTICITY REQUIREMENT

- Resources should be automatically provisioned and released based on demand.
- Pay only for resources actually used.



Example: Video streaming services allocate additional resources during live events.

10 COST EFFICIENCY REQUIREMENT

- Optimize resource utilization to reduce operational costs.
- Select appropriate cloud service models (IaaS, PaaS, SaaS).



Example: Startups often use SaaS solutions to avoid infrastructure costs.

11 COMPLIANCE & GOVERNANCE REQUIREMENT

- Ensure adherence to legal, regulatory, and organizational policies.
- Maintain audit trails and data privacy standards.



Example: Financial institutions must comply with RBI and data protection regulations.

12 MONITORING & MAINTENANCE REQUIREMENT

- Continuously monitor application performance and resource usage.
- Implement logging, alerting, and automated maintenance.

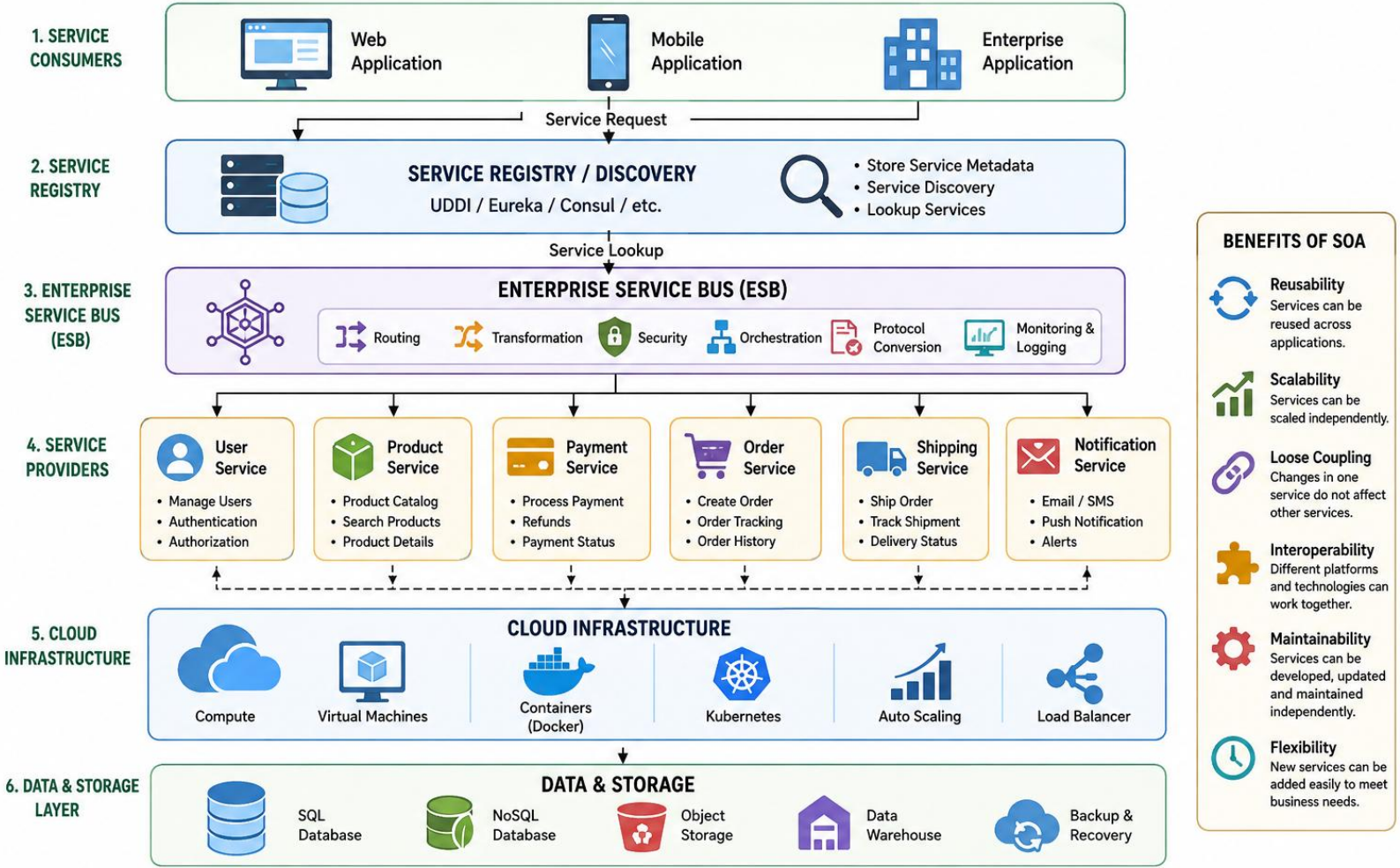


Example: Cloud monitoring tools detect server failures and notify administrators immediately.

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12

Following these requirements helps in designing robust, scalable, secure, reliable, and cost-effective cloud applications.

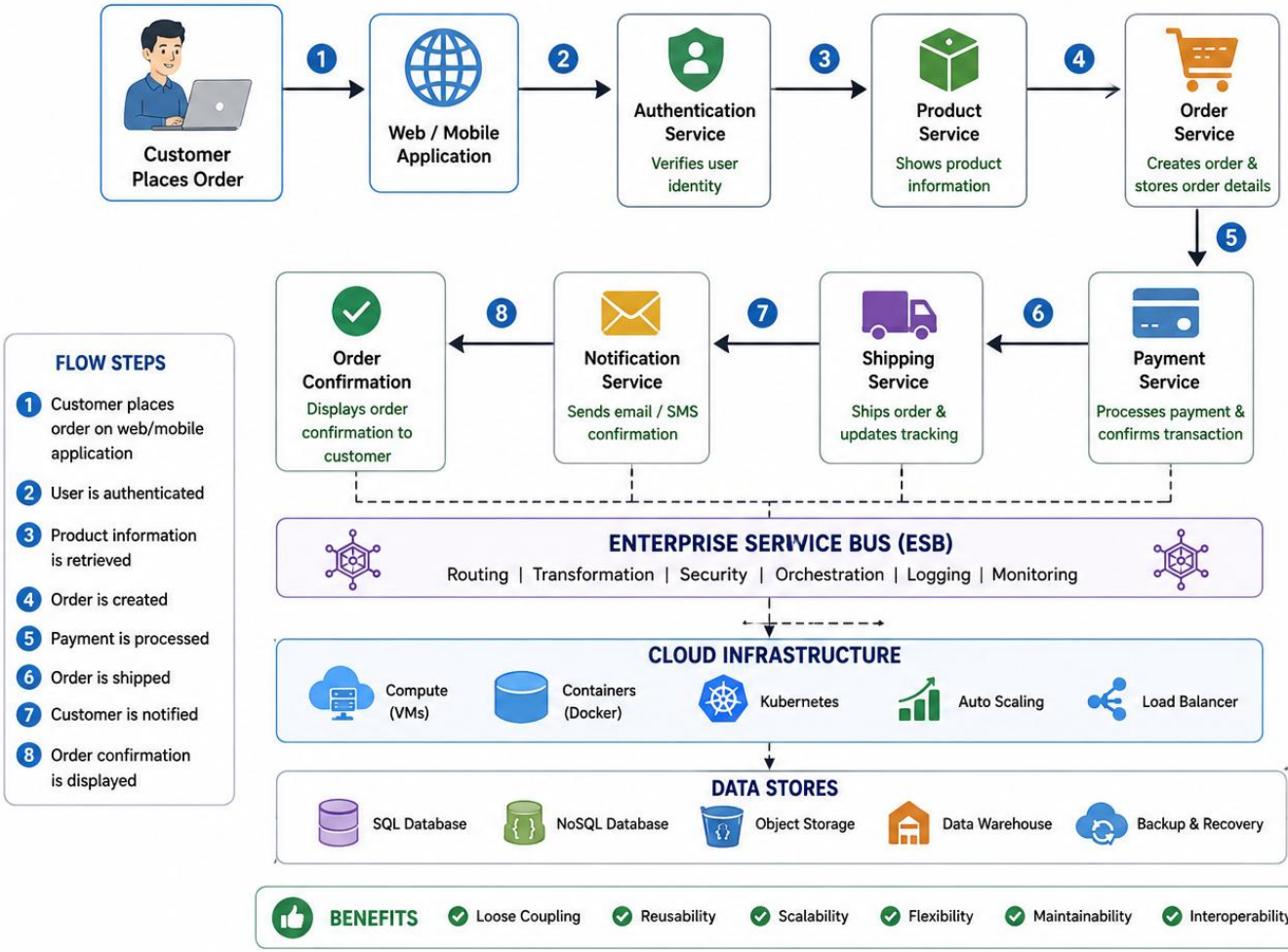
SERVICE-ORIENTED ARCHITECTURE (SOA) FOR CLOUD APPLICATIONS



Service Oriented Architecture (SOA) for Cloud Application

1. **Service Consumer:** The client or application that requests a service. It can be a web application, mobile app, or another service. Example: Customer placing an order through an e-commerce website.
2. **Service Provider:** Creates and provides the service. Publishes service details for users. Example: Payment Service providing online payment functionality.
3. **Service Registry:** Directory where available services are registered. Helps consumers discover services dynamically. Example: UDDI, Eureka Server, Consul.
4. **Enterprise Service Bus (ESB):** Middleware that enables communication among services. Handles routing, transformation, security, and protocol conversion. Functions: Message routing Data transformation Load balancing Security enforcement
5. **Cloud Services:** Each business functionality is implemented as an independent service. Examples: Authentication Service Inventory Service Payment Service Shipping Service Notification Service
6. **Cloud Infrastructure Provides:** Compute resources Storage Networking Virtualization Examples: AWS EC2Microsoft, Azure, Google Cloud Platform

SOA WORKFLOW EXAMPLE (E-COMMRRCE)



Parallelization within Cloud Application

Parallelization is the process of dividing a large computational task into multiple smaller tasks that can be executed simultaneously on multiple processors, virtual machines, containers, or cloud resources. Cloud computing provides virtually unlimited resources, making parallel execution efficient and scalable.

Need for Parallelization in Cloud Applications

- Reduce execution time
- Improve application performance
- Handle large-scale data processing
- Support real-time analytics
- Utilize distributed cloud resources effectively
- Example: Processing millions of customer transactions on an e-commerce platform.

Parallelization within Cloud Application

Step 1: Problem Identification

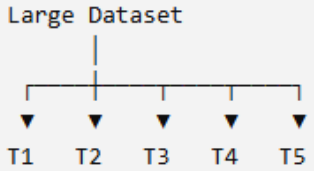
Identify computationally intensive tasks that can be executed simultaneously.

Examples:

- Image processing
- Video rendering
- Big data analytics
- Machine learning training

Step 2: Task Decomposition

Break a large task into smaller independent subtasks.



Example: Divide a 1 TB dataset into five 200 GB chunks.

Parallelization within Cloud Application

Step 3: Resource Allocation

Allocate cloud resources to execute subtasks.

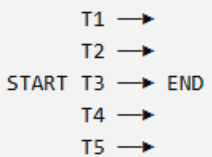
Resources may include:

- Virtual Machines (VMs)
- Containers
- Serverless Functions

```
Task T1 → VM1
Task T2 → VM2
Task T3 → VM3
Task T4 → VM4
Task T5 → VM5
```

Step 4: Parallel Execution

Execute all subtasks simultaneously.



Parallelization within Cloud Application

Step 5: Synchronization

Coordinate and manage dependencies among tasks.

Example:

- Wait until all subtasks complete.
- Handle shared resources.

Step 6: Result Aggregation

Combine outputs from all subtasks.

```
Result1
Result2
Result3
Result4
Result5
   |
   ▼
Final Result
```

Step 7: Delivery to User

Provide the processed result to the application or user.

Insights about Cloud Computing Architecture

- Cloud Architecture refers to the various components in terms of **databases, software capabilities, applications, etc.** which is used by companies of all sizes for leveraging the power of cloud resources to solve all kinds of business problems.
- Even at home, we use cloud technologies for various daily activities. From Microsoft Office 365, Google Docs, and Spotify, these services are provided to us through cloud infrastructure.
- The entire cloud architecture is aimed at providing the users with **high bandwidth**, allowing users to have **uninterrupted access** to data and applications, **on-demand agile network** with possibility to move quickly and **efficiently** between servers or even between clouds and most importantly network security

Principles of Cloud Computing Architecture Design

A complete cloud computing architecture design is also a step-by-step process that begins with collecting requirements and analysis, designing the architecture based on the requirements analysis, evaluating improvements and delivering implementation, and then achieving continuous operations.

To proceed with architecture design based on Cloud Computing, all technical solutions should follow certain principles, which is the target to be hit in architecture design.

There are 6 principles of cloud computing architecture design, including reasonable **deployment, business continuity, elastic expansion, performance efficiency, security compliance, and continuous operation**.

Principle 1: Reasonable Deployment

- The deployment of business systems on the public cloud includes the cloud hosts in the form of virtual machines and physical cloud hosts with **higher performance**. Hosting services include hosted applications and hosted physical servers.
- The cloud computing **operating system** is extracted and packaged as **independent software** and services to be deployed in the user's private environment. Unlike public clouds, which are available to "any" user, private deployments are only available to a select few users.
- Hybrid architecture enables **unified management and scheduling of resources** for public cloud and privatized deployed platforms, traditional VMware, OpenStack virtualization platforms or physical servers.



Principle 2: Business Continuity

Business continuity mainly refers to the 3 aspects of **high availability, continuous operations, and disaster recovery** and the design pattern is developed according to this logic.

- **High Availability** refers to avoiding business interruption by redundancy and other designs when the resources running the business fail.
- **Continuous Operations** mean that the resources for business operation are fault-free and the business can continuously provide services.
- **Disaster Recovery** refers to the ability to restore applications and data in different environments when the business operating environment is damaged.



Principle 3: Elastic Expansion

- Tightly coupled systems are not easy to expand, and it is difficult to troubleshoot problems when software bugs and system failures occur.
- To maintain the elastic expansion of the system, the first step is to decouple the system components, including the decoupling of dynamic data and static data.

Component decoupling is a prerequisite for achieving expandability and can be done in the following ways.

- Remain stateless and store state data in Redis.
- Put in load balancing, as expansion and scaling had no effect on the overall business.
- Decoupled by message queues or API Gateway. Producers and consumers can be expanded without affecting each other.
- Achieve global load balancing of business. Back-end business can be expanded in a hybrid architecture and multi-cloud environment.



Principle 4: Performance Efficiency

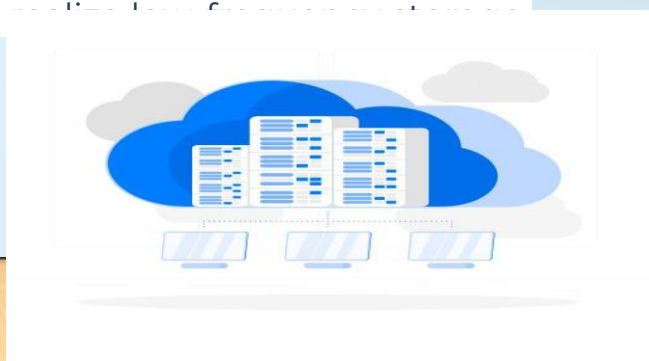
- The primary goal in performance efficiency is to discover and enhance the performance of the application and improve the efficiency of resources and components.
- The first is **computing performance**. The single-machine performance is improved by using high-configuration cloud hosts or physical cloud hosts, and the overall service performance is expanded through clusters.
- The second is **storage and caching**. Cache hot data and store temporary state data by Redis, and performing in-memory calculations can improve business performance.
- The third is the **optimization of network**. Select the optimal data center when the business is deployed globally, and improve the network performance based on global infrastructure network.



Principle 5: Security Compliance

On the one hand, security compliance is to meet the **business security** protection of their own needs, on the other hand, it is to meet the security supervision requirements.In the specific implementation, these two aspects will be crossed together.

- **Set up master accounts and sub-accounts** in the account system, and manage public keys and private keys separately; Set appropriate roles and assign the **minimum permissions** required to accounts and roles.
- **Controlling network access** through ACLs; Restrict open ports on cloud hosts through security groups; Control communication across subnets through subnetting and routing. Configure the database and the cloud hosts that only need internal access to the intranet VPCs, set the VPCs that are allowed to access, and set them to not connect to the extranet.
- Prevent DDoS, cc, SQL injection, XSS and other attacks.
- **Security audit**; Keep access logs and operation logs; Gradually [illegible] and archive storage, etc.



Principle 6: Continuous Operation

- Cloud resources, cloud services, events and users' applications will be monitored in continuous operation, and **alarms will be set**.
- When alarm conditions are reached, relevant personnel will be notified by phone, SMS, email, WeChat, etc.
- Alarms will be handed over to callback functions, which can realize automated fault handling or corresponding contingency plans and reduce manual intervention.
- In addition, it needs to have automatic response and processing functions. Automatic scaling can automatically expand or shrink the number of cloud hosts by monitoring indicators such as CPU.
- Detect changes in consumption and business costs in a timely manner and optimize costs. Set alarm values for account balances to avoid rapid spending and achieve cost control.